CS 3511 - Final Project - Team 7

Joseph Farkas, John McAlinden, Gabriel Nech,
Aesha Shah, Shivali Singh, Joshil Vaddi

April 2026

1 Project Statement & Motivation

Standard binary search trees perform well when balanced, but can degrade to linear time in the worst case when insertions or searches occur in an unfavorable order. Explicit balanced trees like AVL and Red-Black trees solve this by enforcing height constraints to guarantee $O(\log n)$ worst-case operations, but at a cost: they require extra memory for structural metadata and treat every access equally, even though real workloads are rarely uniform.

Splay trees address both limitations. A splay tree is a self-adjusting binary search tree that moves each accessed element to the root via a sequence of rotations called splaying, so frequently used elements naturally rise to the top. Rather than guaranteeing worst-case efficiency per operation, splay trees achieve $O(\log n)$ amortized time across a sequence of operations, along with useful adaptive properties like working-set behavior and static optimality [1]. Their simplicity, memory efficiency, and exploitation of access locality make them an elegant alternative to rigid balanced structures.

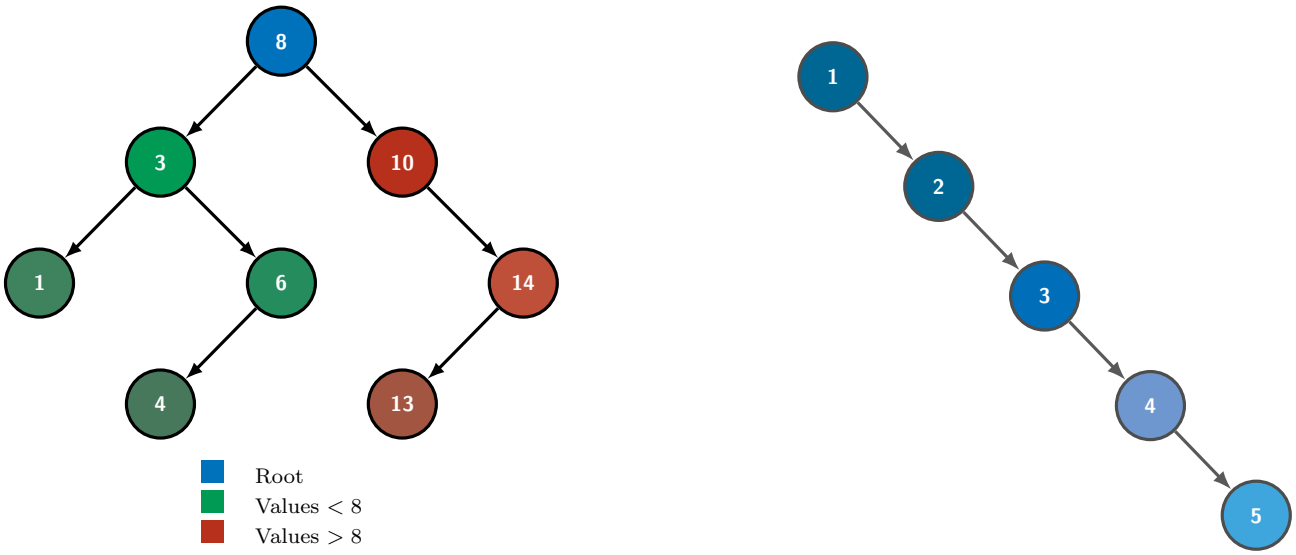
2 Prerequisite Background

2.1 Time Complexity

To begin, we introduce the concept of time complexity, which is to say the measure of how computationally intensive an algorithm might be. Because computation time is greatly dependent on hardware specifics, we tend to compare algorithms by the growth of their time complexity. More specifically, we apply Big O notation to describe an upper bound for a function representing the number of steps needed to compute an algorithm (in a "worst-case scenario"). An algorithm which can be computed in $O(n)$, or linear time, has a computational time which grows at most by some constant multiple of n, where n often represents the size of the input.

2.2 Binary Search Trees

A binary search tree (BST) is a data structure that stores keys such that left subtree keys are smaller and right subtree keys are larger than each node's key (see Figure 1a) [2].



(a) Semantically colored binary search tree.

(b) Right-skewed unbalanced binary search tree.

Figure 1: **Comparison of binary search tree shapes.** Figure a shows a BST with semantically colored nodes indicating the ordering rule relative to the root. Figure b shows an unbalanced BST produced by inserting keys in increasing order.

The structure of binary search trees allows most operations, including search, insertion, and deletion, to be completed in $O(h)$ time, meaning that the computational cost grows at most proportional to h, the tree's height. However, an unproperly managed BST can become *unbalanced* or *skewed*, meaning the height of the tree balloons to nearly n, the number of nodes in the tree, and the tree essentially acts as a linked list (see Figure 1b).

Implementations of BST such as AVL trees and Red-Black trees attempt to "balance" the tree: an effort to minimize the height such that it approaches $\log(n)$. In order to maintain a semblance of balance, tree rotations—which swap a parent-child pair while preserving BST ordering—are used as the core restructuring primitive across AVL trees, Red-Black trees, and even splay trees!

Earlier self-adjusting heuristics, such as single rotation (rotate the accessed node up one level) and move-to-root (rotate it all the way up), are intuitive but insufficient. Certain access sequences (a set of access operations executed back-to-back) can force these methods into poor performance with total costs far exceeding optimal balanced structures [3].

2.3 Amortization and Potential

To better understand the appeal of splay trees, we introduce the notion of amortized complexity. In contrast to standard Big O time complexity analysis, which focuses on highlighting the single worst case an algorithm might face, an amortized analysis measures the average cost per operation over a sequence. For example, when dealing with a dynamic array, like a Python list or a Java ArrayList, the worst-case time complexity of *appending* an item to the list is $O(n)$, since when the list is full, each of the n elements will need to be copied to a new, larger block of memory [4]. However, taking an amortized analysis approach, we note that the vast majority of *appends* take $O(1)$, or constant time, since the item can simply be inserted at the end of the list. In general, amortized cost can be calculated using the aggregate method:

$$\text{Amortized Cost} = \frac{\text{Total Cost of a Sequence of Operations}}{\text{Number of Operations}}$$

Over n append operations, the total copying cost forms a geometric series bounded by $2n$. The appends not requiring a copy take constant time, for a total of up to n operations. Therefore, the total cost of n appends is bound by $3n$, which can be divided by n operations to imply an amortized cost of $O(1)$ per append.

An alternate method for calculating amortized cost is with the concept of potential. Much like potential energy in physical applications, we define a potential function to represent how cheap operations build up potential, which helps compensate for a rare, expensive operation.

More formally, we assign the structure a stored energy Φ. An operation with actual cost t, Φ potential before the operation, and Φ' afterwards has amortized cost

$$a = t + \Phi' - \Phi$$

Potential is especially powerful, as it allows us to estimate the total time of a sequence of m operations given the final and initial potential, where t_j is the actual cost of any given operation, and a_j is the amortized cost [1].

$$\sum_{j=1}^{m} t_j = \sum_{j=1}^{m} (a_j + \Phi_{j-1} - \Phi_j) = \sum_{j=1}^{m} a_j + \Phi_0 - \Phi_m$$

In cases where $\Phi_m \geq \Phi_0$, the total amortized time is an upper bound on the total actual time. This framework will allow us to explain how splay trees remain efficient long-term despite lacking explicit balance constraints—each restructuring step improves future access times by pulling useful (commonly accessed) nodes toward the root.

3 Algorithm Details

3.1 Splay Trees

Following from the structure of the binary search trees, the splay tree dynamically adapts to access patterns by migrating recently used nodes towards the root. This self-adjusting behavior is made possible by a process called **Splaying** [3, 1].

After a node is accessed, it is slowly rotated up to the root through **Zig, Zig-Zig, and Zig-Zag** unit operations.

Case 1: Zig The accessed node is rotated up to the root.

- If rotating right, the previous parent adopts the right subchild of the accessed node.
- If rotating left, the previous parent adopts the left subchild of the accessed node.



Figure 2: Right Rotation Zig Example

Case 2: Zig-Zig The accessed node and parent are both left or both right children. This configuration triggers two rotations in the same direction.

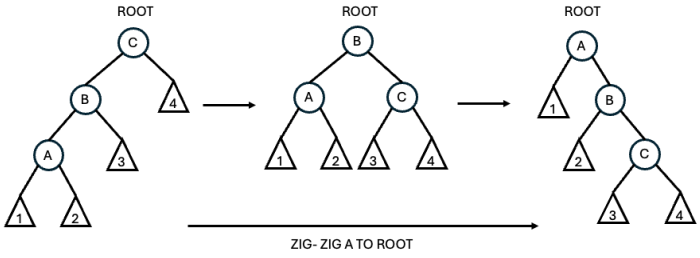


Figure 3: Right Rotation Zig-Zig Example

Case 3: Zig-Zag The accessed node is a left child and the parent is a right child (or vice-versa). In the example shown in Figure 4, a leftwards zig is performed on B, followed by a rightwards zag to rotate A to the root.

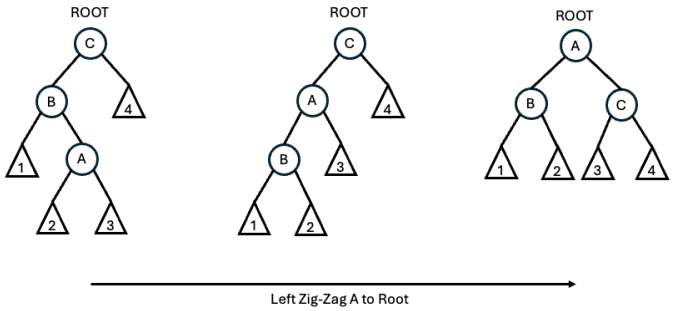


Figure 4: Zig-Zag Example

3.2 Update Operations on Splay Trees

- *access*(i, t): This operation searches for item i in tree t. If i is present, it returns a pointer to its location; otherwise, it returns a pointer to the null node.
  - Search down from the root of t looking for i. If the search reaches a node x containing i, splay at x and return a pointer to it.
  - If the search reaches a null node, splay at the last nonnull node reached (the node from which the search ran off the bottom of the tree) and return a pointer to null.

Most update operations in splay trees are conceptually simplified by first implementing the *join* and *split* primitives.

- *join*(t_1, t_2): This operation combines two trees, t_1 and t_2, into a single tree. It assumes that all items in t_1 are less than all items in t_2.
  - Access the largest item i in t_1. After splaying at i, the root of t_1 contains i and has a null right child. The *join* is completed by making t_2 the right subtree of this root.
- *split*(i, t): This operation constructs two trees, t_1 (containing items in $t \leq i$) and t_2 (containing items in $t > i$).
  - Perform *access*(i, t) and then break either the left or right link from the resulting root, depending on whether the root's item is greater than i or not.

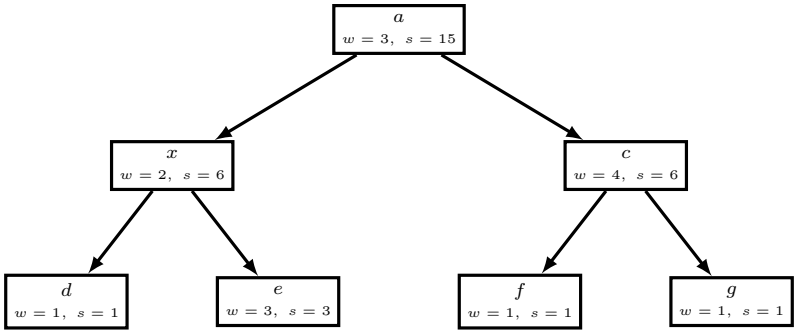
Using these primitives,

- *insert*(i, t): Adds item i to tree t, assuming it is not already present.
  - Perform *split*(i, t) and replace t with a new root node containing i. The trees t_1 and t_2 returned by the *split* become the left and right subtrees of this new root.
  - Alternatively, search for i, replace the null pointer reached during the search with a new node containing i, and then *splay* the tree at that new node.
- *delete*(i, t): Removes item i from tree t, assuming it is present.
  - Perform *access*(i, t), which moves the node containing i to the root. Replace t by performing a *join* of the root's left and right subtrees.
  - Alternatively, search for the node x containing i. Replace x as a child of its parent y with the *join* of the left and right subtrees of x, and then *splay* at y.

3.3 Access Lemma

The access lemma is a foundational property of splay trees that empowers their amortized complexity advantages. We seek to prove an upper bound on the amortized time to splay a tree, where the number of steps is taken to be the number of rotations employed by the algorithm.

We begin by defining properties of splay trees. Take each item i stored in the tree to have a positive, fixed, and arbitrary weight given by $w(i)$. We also define the size $s(x)$ of a node x of the tree as the sum of the weights for all items within the subtree given by the root x. Then, take the rank $r(x)$ for each node sx to be $\log s(x)$ (where log is the binary logarithm). For an example of these tree properties, see Figure 5.



$$s(x) = 2 + 1 + 3 = 6, \qquad r(x) = \log_2 6, \qquad \Phi = \sum_{\text{nodes } y} r(y)$$

Figure 5: Example weighted tree.

Now, we can define the potential function for a tree to be the sum of the ranks for each node in the tree. More formally, for a tree with n nodes.

$$\Phi = \sum_{j=1}^{n} r(x)$$

We can now introduce our lemma

Lemma 1 (Access Lemma): The amortized time to splay a tree with root t at a node x is at most

$$3\big(r(t) - r(x)\big) + 1 = O\left(\log\left(\frac{s(t)}{s(x)}\right)\right).$$

Proof: To prove this, we consider the various steps needed to splay any given tree.

First, notice that if no rotations are needed to splay the tree (the tree is already in the desired state), then the lemma clearly holds, since the amortized time will be constant (simply checking the root of the tree).

Now consider any splaying step. We define s and s' as the size before and after the step, respectively, and r and r' as the rank before and after. We also define y as the "parent" or x and z the parent of y (if it exists) before the splay step. We proceed to evaluate the amortized cost of each of the three possible splaying steps using the potential amortization method introduced in Section 3.3.

Case 1: Zig. In this case, one rotation is completed, so the actual cost of the operation is taken as 1. The change is potential is given by the difference in rank for x and y.

$$\begin{aligned}
a &= t + \Phi' - \Phi && \text{definition of amortization by potential} \\
&= 1 + r'(x) + r'(y) - r(x) - r(y) && \text{since only } x \text{ and } y \text{ change rank} \\
&\le 1 + r'(x) - r(x) && \text{since } r'(y) \ge r(y) \\
&\le 1 + 3\big(r'(x) - r(x)\big) && \text{since } r'(x) \ge r(x)
\end{aligned}$$

Case 2: Zig-Zig. Two rotations are done, so the actual cost is taken as 2. The change is potential is given by the difference in rank for x, y, and z.

$$\begin{aligned}
a &= t + \Phi' - \Phi && \text{definition of amortization by potential} \\
&= 2 + r'(x) + r'(y) + r'(z) - r(x) - r(y) - r(z) && \text{since only } x, y, \text{ and } z \text{ change rank} \\
&= 2 + r'(y) + r'(z) - r(x) - r(y) && \text{since } r'(x) = r(z) \\
&\le 2 + r'(x) + r'(z) - 2r(x) && \text{since } r'(x) \ge r'(y) \text{ and } r(y) \ge r(x)
\end{aligned}$$

We now seek to prove the last sum is at most $3\big(r'(x) - r(x)\big)$, which we can do by proving the difference between the bound and the sum is non-negative. In other words, $2r'(x) - r(x) - r'(z) \ge 2$, which is equivalent to $r(x) + r'(z) - 2r'(x) \le -2$

The log function is convex, implying that $\log x + \log y$ for $x, y > 0$ and $x + y \le 1$ has a maximum value of -2, which is achieved when $x = y = \frac{1}{2}$. We can rewrite our expression using properties of logarithms as

$$r(x) + r'(z) - 2r'(x) = \log\big(\frac{s(x)}{s'(x)}\big) + \log\big(\frac{s'(z)}{s'(x)}\big) \le -2$$

which we can show is true, since $s(x) + s'(z) \le s'(x)$ and $s(x), s'(z) \ge 0$.

Thus, the amortized time in case 2 is at most

$$3\big(r'(x) - r(x)\big).$$

Case 3: Zig-Zag. Two rotations are performed, so the actual cost is again taken as 2. The change in potential is given by the different in the rank for x, y, and z.

$$
\begin{aligned}
a &= t + \Phi' - \Phi && \text{definition of amortized cost using potential} \\
&= 2 + r'(x) + r'(y) + r'(z) - r(x) - r(y) - r(z) && \text{since only } x, y, \text{ and } z \text{ can change rank} \\
&\le 2 + r'(y) + r'(z) - 2r(x) && \text{since } r'(x) = r(z) \text{ and } r(x) \le r(y)
\end{aligned}
$$

We now show that this final quantity is at most $2\big(r'(x) - r(x)\big)$. Equivalently, it is enough to prove that $2r'(x) - r'(y) - r'(z) \ge 2$. This follows by the same argument as in Case 2, using the inequality

$$s'(y) + s'(z) \le s'(x).$$

Therefore,

$$a \le 2\big(r'(x) - r(x)\big).$$

Since

$$2\big(r'(x) - r(x)\big) \le 3\big(r'(x) - r(x)\big),$$

the amortized time in Case 3 is at most

$$3\big(r'(x) - r(x)\big).$$

Thus, each step in splaying takes at most amortized time $3\big(r'(x) - r(x)\big) + 1$, we can then sum across all steps Summing the amortized bounds over all splaying steps gives the total amortized cost c for splaying the tree

$$c \;\le\; 3\sum_i \big(r_i'(x) - r_i(x)\big) + 1.$$

The $+1$ accounts for the possible final zig step. The rank differences telescope, since the rank of x after one step is its rank before the next, so

$$\sum_i \big(r_i'(x) - r_i(x)\big) = r'(x) - r(x).$$

Hence the total amortized cost is at most

$$3\big(r'(x) - r(x)\big) + 1.$$

After splaying, x is the root, so $r'(x) = r(t)$, and therefore the bound becomes

$$3\big(r(t) - r(x)\big) + 1 = O(\log(s(t) - \log(s(x)) = O\left(\log\left(\frac{s(t)}{s(x)}\right)\right)$$

3.4 Performance Theorem

Having established the Access Lemma, we can now state the central performance guarantee of splay trees.

- **Performance Theorem:** A sequence of m operations on a splay tree containing n items, starting from an empty tree, runs in total time

$$O((m+n)\log(m+n)).$$

Justification. Assign every item i the uniform weight $w(i) = 1/n$. Then $s(t) = 1$ for the root, so $r(t) = \log 1 = 0$, and the initial potential satisfies $\Phi_0 \le n \log n$. By the Access Lemma (Lemma 1), the amortized cost of each splay operation is at most

$$3\big(r(t) - r(x)\big) + 1 \;=\; O(\log n).$$

Summing over all m operations and accounting for the telescoping of potential gives a total actual cost of

$$\sum_{j=1}^{m} t_j \;\le\; \sum_{j=1}^{m} a_j + \Phi_0 - \Phi_m \;\le\; O(m \log n) + n \log n \;=\; O((m+n)\log(m+n)).$$ □

3.5 Applications of the Access Lemma

The Access Lemma is powerful precisely because the weight assignment $w(i)$ is arbitrary. By choosing different weight functions, we obtain the following corollaries, each capturing a distinct adaptive property of splay trees.

3.5.1 Static Optimality

- **Static Optimality:** Let $q_i > 0$ denote the number of times item i is accessed in a sequence of m operations, with $\sum_i q_i = m$. Then the total cost of all accesses in a splay tree is

$$O\left(m + \sum_{i=1}^{n} q_i \log \frac{m}{q_i}\right).$$

Justification. Set $w(i) = q_i$. Then $s(t) = m$ and, for each access to item i, the Access Lemma gives amortized cost $O(\log(m/q_i))$. Summing q_i accesses to item i over all i yields the stated bound. □

The bound $\sum_i q_i \log(m/q_i)$ matches, up to a constant factor, the cost of the **optimal static** search tree built with full knowledge of the access frequencies in advance. That is, a splay tree automatically matches the best fixed tree for any access distribution, with no prior knowledge required.

3.5.2 Working-Set Theorem

- **Working-Set Theorem:** Let $t(j)$ denote the number of **distinct** items accessed since the last access to item x_j (i.e. the "working-set size" at access j). Then the total cost of a sequence of m accesses is

$$O\left(m + n\log n + \sum_{j=1}^{m} \log\big(t(j)+1\big)\right).$$

Justification. Number the accesses $1, \ldots, m$. Consider the fixed set of weights

$$1,\ \frac{1}{4},\ \frac{1}{9},\ \ldots,\ \frac{1}{n^2},$$

assigned to items in order of recency (most recently accessed items receive larger weights). After each access, these weights are reassigned so that the accessed item becomes the most recent.

Under this scheme, just before access j, the item x_j has weight $1/(t(j)+1)^2$, and the total weight is $O(1)$ since $\sum_{k=1}^{n} 1/k^2 = O(1)$. Applying the Access Lemma, the amortized cost of access j is therefore

$$O\Big(\log \frac{W}{w(x_j)}\Big) = O\big(\log\big((t(j)+1)^2\big)\big) = O\big(\log\big(t(j)+1\big)\big).$$

Summing over all accesses and adding the $O(n\log n)$ initial potential gives the result. □

The working-set property captures **temporal locality**: if the same small set of items is accessed repeatedly, those items naturally rise toward the root and subsequent accesses to them are served cheaply.

3.5.3 Dynamic Finger Theorem

- **Dynamic Finger Theorem:** Let $d(j) = |x_j - x_{j-1}|$ be the difference in key rank between consecutive accesses. Then the total cost of a sequence of m accesses is

$$O\left(m + n\log n + \sum_{j=2}^{m} \log\big(d(j)+1\big)\right).$$

Justification. Assign item i a weight that decreases geometrically with its distance from the previously accessed item: $w(i) \propto 2^{-|i-x_{j-1}|}$. The total weight is $O(1)$, while the weight of x_j is $2^{-d(j)}$, so $s(t)/s(x_j) = O(2^{d(j)})$ and the Access Lemma yields amortized cost $O(\log(d(j)+1))$. Making this argument fully rigorous requires re-weighting at each step [5]; □

The dynamic finger property captures **spatial locality**: if successive accesses cluster around nearby keys, each access costs nearly $O(1)$, reflecting that a splay tree reorganizes itself around the "finger" of recent activity.

Practical Applications of Splay Trees

The primary advantage of a splay tree is its ability to adapt to nonuniform access patterns, making it ideal for systems where a small subset of data is used frequently. Because the splaying process moves the most recently accessed items to the root, frequently accessed data becomes progressively cheaper to retrieve.

- **File Systems:** Windows NT utilizes splay trees to manage file structures. In this context, filenames serve as keys and disk locations as values; the tree ensures that frequently accessed files are moved closer to the root for faster subsequent lookups [6].
- **String Editors:** The Unix utility `sed` utilizes splay trees to store strings and their associated transformation rules. This ensures that the most commonly applied rules are positioned near the top of the tree, making them the cheapest to access during line-by-line processing [6].
- **Network Optimization:** Splay trees are used to implement "link/cut trees," a specialized data structure for maintaining a forest of rooted trees. These are essential for efficiently solving network problems, such as maximum flow and minimum cost flow algorithms.
- **Memory Management:** Splay trees are well-suited for implementing first-fit storage allocation. They offer an amortized time bound of $O(\log n)$ for both allocation and deallocation operations and may outperform traditional methods like the "roving pointer" in practice.
- **Data Compression:** Splay trees can simplify lexicographic or multidimensional search trees, which are often used in string sets and digital tries to improve repeated access efficiency.

4 Conclusion

Splay trees represent a paradigm shift in data structure design, moving away from the rigid, proactive balancing seen in AVL or Red-Black trees toward a reactive, self-adjusting approach. By utilizing the splay operation as a core primitive, these trees ensure that frequently accessed data remains near the root, effectively optimizing themselves for non-uniform access patterns.

Through the lens of the Access Lemma [3], we have shown that while individual operations may occasionally be expensive, the amortized cost remains $O(\log n)$, providing a performance guarantee that rivals explicitly balanced structures. Furthermore, the Static Optimality and Working-Set theorems highlight the splay tree's remarkable ability to exploit temporal and spatial locality without requiring prior knowledge of the workload [5, 1].

Ultimately, the splay tree stands as an elegant solution for systems where memory overhead is a concern and access patterns are skewed. By balancing simplicity in implementation with sophisticated amortized efficiency, splay trees remain a fundamental tool in the study of adaptive algorithms and modern computational complexity [2].

References

[1] Robert Endre Tarjan. *Amortized Computational Complexity*. SIAM, 1985.

[2] Thomas H. Cormen et al. *Introduction to Algorithms*. 4th ed. MIT Press, 2022.

[3] Daniel Dominic Sleator and Robert Endre Tarjan. "Self-adjusting binary search trees". In: *Journal of the ACM* 32.3 (1985), pp. 652–686.

[4] Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser. *Data Structures and Algorithms in Java*. 6th ed. Wiley, 2014.

[5] Richard Cole et al. "On the dynamic finger conjecture for splay trees". In: *SIAM Journal on Computing* 30.1 (2000), pp. 1–25.

[6] Justin Wyss-Gallifent. *CMSC 420: Splay Trees*. Tech. rep. University of Maryland, 2023.